

NP-Complete Infeasibility Experiment:

MODULE 6 CODING ASSIGNMENT- SEJAL PATIL

Background

The Module 6 assignment solved Bipartite Matching using Max-Flow in $O(nm)$ time. The brute-force alternative checks every possible subset of the m edges, giving a running time of 2^m . With 100 engineers and 300 engineer-project links ($m = 300$), brute force requires on the order of 2^{300} basic operations. This assignment measures, experimentally, how long that actually takes, and why exponential growth makes it infeasible no matter how much hardware is thrown at it.

Experimental setup

A simple counting loop was run in Python and timed with `time.perf_counter()`:

```
cnt = 0
for i in range(1,000,000):
    cnt += 1
```

The loop was run 5 times to get a stable average. Individual trial times, in seconds: 0.096725, 0.061710, 0.060949, 0.058073, 0.058023.

Average time to count from 1 to 1,000,000: 0.067096 seconds.

All scaling below uses the ratio method suggested in the assignment: $\text{time}(N) = \text{time}(1,000,000) \times (N / 1,000,000)$. Value for a final result is in decimal.

Part 1: Time to count from 1 to 2^{300} on a laptop

The exact value of 2^{300} is

2037035976334486086268445688409378161051468393665936250636140449354381299
763336706183397376

Scaling the measured rate: $\text{time} = 0.067096 \text{ seconds} \times (2^{300} / 1,000,000)$.

Total time in seconds=

1366769658681386784442676319095156370939093233414096586726824795898815676
88920839638.081230340096

Converted to years=

4331031696584616017829861330060449371749097629141939142161713171783708763
940.250197672865818 years

Approximately, it will be

4,331,031,696,584,616,017,829,861,330,060,449,371,749,097,629,141,939,142,161,713,171,783,708,763,940 years, 91 days, 9 hours, 13 minutes, 58 seconds.

For comparison, the age of the universe is about 13,800,000,000 years which is an 11-digit number. The time required for brute force above is a 76-digit number, roughly 65 digits (65 orders of magnitude) longer. In other words, brute-force checking every subset of 300 edges would take a number of years so much larger than the age of the universe that the difference itself cannot be pictured; it is not simply 'slow,' it is a different category of number entirely.

Part 2: Time to count from 1 to 2^{301}

The exact value of 2^{301} is

4074071952668972172536891376818756322102936787331872501272280898708762599526673412366794752

Total time in seconds as an 84-digit whole number is

273353931736277356888535263819031274187818646682819317345364959179763135377841679276.162460680192

Converted to years as in 76-digit whole number is

8662063393169232035659722660120898743498195258283878284323426343567417527880.500395345731636 years

Adding just one more edge ($m = 300$ going to 301) exactly doubled the running time — the year value for 2^{301} is precisely twice the year value for 2^{300} . This is the defining signature of exponential growth: every single unit increase in problem size doubles the total work, no matter how large the problem already is. A linear or polynomial-time algorithm never behaves this way; one extra edge there would add only a small, roughly constant increment of work, not double the entire computation.

Part 3: Time to count to 2^{300} on a 1,000,000-GPU farm

Assume 1,000,000 GPUs, each 1,000 times faster than the laptop used above. The combined speedup is $1,000,000 \times 1,000 = 1,000,000,000$ — one billion times faster than the original laptop.

Total time in seconds=

1366769658681386784442676319095156370939093233414096586726824795898815676
88.920839638081230

Converted to years=

4331031696584616017829861330060449371749097629141939142161713171783.70876
3940250197672866 years

Converted to months (years x 12) in a 68-digit whole number=

51972380359015392213958335960725392460989171549703269705940558061404.5051
67283002372074390

Billion-fold hardware speedup does not solve the problem here. Dividing the original 76-digit year value by 1 billion only removes 9 digits, leaving a 67-digit number of years i.e., still about 56 more digits than the 11-digit age of the universe. The result is nowhere close to being countable in months, years, or even many repeated lifetimes of the universe. Thus, a billion-fold speedup is a rounding error against a 2^{300} -sized exponential.

Part 4: Why parallel hardware can't rescue exponential algorithms?

Buying more machines increases computing power linearly: k machines gives roughly a k fold speedup at its best. But the brute-force cost of an NP-complete problem grows exponentially with input size (2^m). Since 2^m grows faster than any linear or polynomial function of available hardware, no realistic amount of parallel hardware can keep pace with it. Each additional unit of problem size doubles the required work, while each additional machine only adds a constant multiple of speed. As Part 3 shows, even a billion-fold hardware speedup barely changes the picture, because the exponential term dominates everything else once m is large.

Some directions that are actually used in practice instead of 'buy more hardware':

1. Use a polynomial-time algorithm for the exact problem when one exists. This is exactly why the Module 6 assignment uses Max-Flow ($O(nm)$) instead of brute force (2^m) for Bipartite Matching. The right algorithm is what turns an infeasible problem into a practical one.
2. Approximation algorithms: for many NP-complete problems, a solution provably within a small factor of optimal can be found in polynomial time, which is often good enough in practice.
3. Heuristics and metaheuristics (greedy methods, local search, simulated annealing, genetic algorithms): no optimality guarantee, but they routinely find very good solutions quickly on real-world instances.

4. Restricting the problem or exploiting structure: parameterized complexity, special-case polynomial algorithms for restricted graph classes, or reducing the effective input size can make specific instances tractable even though the general problem is not.
5. Quantum computing: quantum computers do not turn NP-complete problems into polynomial-time problems in general. Grover's algorithm gives only a quadratic speedup for unstructured search, which helps but still leaves exponential growth intact. Shor's algorithm gives a much larger speedup, but only for a specific structured problem, integer factorization, not for NP-complete problems in general. Quantum computing narrows the gap for some problems; it is not a general escape from exponential blow-up.
6. Accept that exact solutions are infeasible at real-world scale and settle for solutions that are good enough and fast enough. This is the practical stance taken by essentially every real system that touches an NP-complete problem, such as scheduling, routing, or matching.

Conclusion

Counting to 2^{300} takes a 76-digit number of years on an ordinary laptop, vastly more digits than the 11-digit age of the universe. Adding a single edge (2^{300} to 2^{301}) exactly doubles that time. Even a billion-fold hardware speedup (a million GPUs, each 1,000 times faster than a laptop) only removes 9 digits from that number, leaving a 67-digit number of years, still utterly infeasible. This is the practical meaning of 'NP-complete problems are infeasible for large inputs: exponential growth outpaces any amount of linear improvement in computing power, which is why algorithm design (such as Max-Flow instead of brute force) and approximation or heuristic methods are the real tools for handling these problems at scale.

Summary table

Scenario	Digits in year value	Feasible in months?
Part 1: Laptop, 2^{300}	76-digit number of years	No
Part 2: Laptop, 2^{301}	76-digit number of years (exactly double Part 1)	No
Part 3: 1,000,000-GPU farm, 2^{300}	67-digit number of years / 68-digit number of months	No
Reference: age of the universe	11-digit number of years (about 13.8 billion)	-